

Compléments / Aide à la mise en oeuvre

Sur la base des systèmes **masse-ressort 1D** vus en TP, plusieurs développements peuvent être envisagés, en prolongement de certaines propriétés et/ou défauts de ces modèles.

Avant de détailler le travail qui vous sera demandé (projet), voilà quelques éléments complémentaires pour vous aider à appréhender ces modèles, faciles à écrire, mais parfois difficiles à utiliser. Cela correspond plus ou moins à ce qui était au menu de la première séance de TP.

Caractéristiques des modèles de départ

- schéma d'intégration numérique d'ordre 2 : **LeapFrog**, potentiellement instable.
- construction modulaires de type "assemblage" de briques élémentaires (masses/ressorts)
- deux types de briques (**PMat** et **Link**), déclinables en un nombre quelconque de modules physiques élémentaires (particule / point-fixe / ressort / frein / liaison-conditionnelle ...)
- modèle élémentaire construit (corde) : système à topologie fixe.
- aucun modèle de gestion de collision.
- **Moteur Physique** : simulateur générique "bas-niveau" :
 - ① calcul et distribution de toutes les forces circulant dans le système entre 2 pas de simulation
 - ② mise à jour de toutes les variables d'états (vit./pos.) par intégration numérique (LeadFrog)
- **Moteur de rendu** totalement indépendant du **Moteur Physique**. Ils se contentent de partager certaines données (états des particules).

Particules ou Masses-Ressorts

Fondamentalement, c'est la même chose. La différence réside dans les interactions entre les masses.

- **Masses-Ressorts** : les éléments matériels (masses) sont ponctuels et reliés par un réseau de liens (ressorts) dont la topologie est "fixe" (comme la corde ou le drapeau).

Ce sont ces liens qui définissent l'espace entre les masses et donnent une forme et une structure aux objets.

La complexité du système dépend de la topologie (nombre de liens), mais reste en général linéaire (par rapport au nombre de masses).

👉 **utilisation classique** : solides déformables ⁽¹⁾. Les particules peuvent bouger mais gardent une organisation (voisinage) constante définie par le maillage de liaisons. Ce maillage est généralement formé de plusieurs "couches" pour assurer la consistance mécanique du modèle. Chaque "couche" correspond à un *tenseur* mécanique (déformation/courbure/torsion).

(1). cf. [wikipédia : Mécanique des Solides Déformables](#)

- **Particules** : elles sont souvent présentées comme des "masses ponctuelles entourées d'un champs de force" (SPH : Smooth Particles Hydrodynamics)⁽²⁾. Dans ces modèles les interactions entre 2 particules sont modélisées par l'intersection des deux champs de force : ce sont les particules elles-mêmes qui portent les lois de transfert d'énergie.

Mais on peut faire exactement la même chose avec un système **Masses-Ressorts** entièrement connecté : chaque masse est reliée à toutes les autres mais les liaisons sont "conditionnelles" (actives seulement lorsque la distance entre les masses est inférieure à un certain seuil – qui définit la "taille" des particules).

Le formalisme de base est exactement le même que pour une modèle de solide déformable, mais les interactions (liaisons) doivent être peu plus élaborées que de simples ressorts.

La complexité est alors en n^2 pour n particules, et le recours à des structures accélératrices (subdivision de l'espace) devient vite nécessaire (idem pour les SPH).

Modèles 1D → 2D → 3D

Les modèles en dimension 1 restent très limités pour des rendus graphiques (ils sont en revanche bien suffisants pour de la synthèse sonore).

Pour des applications graphiques, il faut au minimum passer en dimension 2 ou 3. La structure générale reste exactement la même. Ce qui change c'est les caractéristiques des composants de base (2 ou 3 coordonnées), et les calculs associés :

- les calculs de forces, vitesses... deviennent vectoriels $\Rightarrow$ 2 informations : module et direction
- les calculs de distances $\Rightarrow$ distance euclidienne $d(A,B) = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$
- les coordonnées *physiques* sont les mêmes que les coordonnées *graphique*
- ces coordonnées sont d'ailleurs (a priori) les seules données communes aux deux moteurs (physique et graphique).
- moteur de rendu 3D $\Rightarrow$ utiliser la lib `<libg3x>` (ou directement OpenGL, ou autre chose...)

Pour tester des **Systèmes de Particules**, la version 2D est bien suffisante et beaucoup plus simple à gérer (structures accélératrices). Pour des **Systèmes Masses-Ressorts**, la version 3D est plus intéressante.

Modularité

Les exemples donnés en TP (en dimension 1) ne sont qu'une façon de faire parmi bien d'autres. Elle présente néanmoins un avantage certain : la modularité algorithmique.

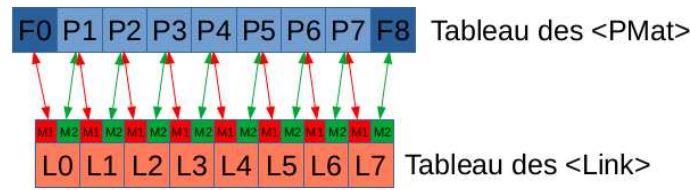
Cette modularité permet de construire des modèles génériques et homogènes, rend le moteur physique indépendant du modèle (exécute en aveugle) et permet une parallélisation efficace (*multithreads*).

Elle s'exprime de 2 façons :

- ① **modularité de construction** : le système repose sur 2 classes d'objets, les "masses" (`<PMat>`) et les "liaisons" (`<Link>`), et peut être vu comme un graphe :
 - une masse ("noeud" du graphe) ignore totalement son environnement. Elle se contente d'accumuler des informations (ici, des forces) via les liaisons qui la connaissent, puis de mettre à jour son état, toujours indépendamment de tout le reste.

(2). cf. [wikipedia : SPH](#)

- une liaison ("arc" du graphe) ne peut exister dans la simulation que si elle est connectée à 2 "masses" : elle récupère les états de ses masses, calcule une force, et la distribue sur ses masses. Chaque liaison est donc totalement indépendante des autres.



structure algorithmique de "corde" : 2 points fixes, 7 particules, 8 liaisons

② **modularité d'exécution** : le modèle final se réduit à deux tableaux (un pour chaque type de brique, masse ou liaison).

☞ Sauf cas particulier (cf. beaucoup plus loin), ces tableaux n'ont à être soumis à aucun ordre précis : toutes les briques peuvent être stockées "en vrac", quelle que soit leur nature.

Le "moteur" de simulation se contente alors d'exécuter séquentiellement, et indépendamment de tout ordre, l'ensemble des algorithmes de chaque module de liaison (distribution des forces élémentaires), puis l'ensemble des mises à jours des états des modules de masse (double intégration temporelle).

L'indépendance des traitements de chaque module permet une parallélisation efficace : chaque processeur (ou *thread*) gère une portion de chacun des tableaux. Seule contrainte : les *threads* gérant les masses doivent attendre que les *threads* gérant les liaisons se soient tous terminés (plus de circulation de force).

attention : cette modularité à l'exécution n'est possible qu'avec des schémas d'intégration explicites (ou semi-explicites, comme **LeapFrog**). Avec un schéma tel que **Euler Implicite**, ce n'est plus possible (ou plus difficile) car la mise à jour des états individuels doit prendre en compte l'intégralité de l'état du système.

Jeu de construction

La classe d'objets <PMat> restera toujours très modeste : elle contient a priori seulement un objet, la *particule* mobile, et sa version "dégénérée", le *point fixe* (particule de masse infinie ou particule immobile).

On peut inventer des objets de type <PMat> plus exotiques (cf. plus loin) tant qu'ils respectent deux règles de base : un objet <PMat> peut constituer un modèle simulable à lui seul, et si il est isolé de toute influence extérieure il ne doit pas bouger (pas de circulation d'énergie interne).

☞ un modèle constitué uniquement de <PMat>, sans aucune liaison, doit être simulable (i.e. il ne doit pas faire "planter" le moteur).

La classe <Link> en revanche peut être aussi fournie qu'on le souhaite et contenir toutes sortes de liaisons plus ou moins complexes et plus ou moins "exotique" (i.e. on peut s'éloigner de toute réalité physique – cf. modèles de type *boids*).

Toute contiennent au minimum deux références vers des points matériels M_1 et M_2 **connectés** et un algorithme renvoyant une paire de forces (une pour chaque M). Ces deux forces sont généralement opposées ($\vec{F}_1 + \vec{F}_2 = \vec{0}$) dans les "vrais" modèles physiques.

Principales liaisons utiles (mécanique très très élémentaire!)

- le **ressort de Hook** ⁽³⁾ : produit une force proportionnelle à son allongement.
 - paramètres : k (raideur, en N/m), l_0 (longueur à vide, calculée à la connexion, sur les positions initiales de M_1 et M_2)
 - force calculée : $\vec{F} = -k * (d - l_0) * \frac{\overrightarrow{M_1 M_2}}{d} = -k * (1 - \frac{l_0}{d}) * \overrightarrow{M_1 M_2}$ où $d = \|\overrightarrow{M_1 M_2}\|$
☞ si d devient trop faible (surcompression), le système peut devenir instable.
 - la masse M_1 reçoit la force $(+\vec{F})$ alors que M_2 reçoit $(-\vec{F})$
- le **frein cinétique** : produit une force proportionnelle à la vitesse relative des 2 masses.
 - paramètres : z (viscosité)
 - force calculée : $\vec{F} = -z * (\vec{V}_{M_2} - \vec{V}_{M_1})$
 - la masse M_1 reçoit la force $(+\vec{F})$ alors que M_2 reçoit $(-\vec{F})$
- le **ressort-frein** : combinaison (montage "parallèle") des deux précédents.
 - paramètres : k (raideur), z (viscosité), l_0 (longueur à vide)
 - force calculée : $\vec{F} = -k * (1 - \frac{l_0}{d}) * \overrightarrow{M_1 M_2} - z * (\vec{V}_{M_2} - \vec{V}_{M_1})$
 - la masse M_1 reçoit la force $(+\vec{F})$ alors que M_2 reçoit $(-\vec{F})$
- les **liaisons conditionnelles** : n'importe quelle liaison, agrémentée d'un test qui la rend active ou inactive. Quelques exemples utiles :
 - la liaison de type "inter-particules" : c'est par exemple un simple ressort-frein dont la force n'est calculée que si la distance entre les masses est inférieure à un certain seuil. Ce seuil d'interaction définit la "taille" des particules. Il peut être un paramètre de la liaison ou être calculé à partir des données lues dans les particules.
 - la liaison de rupture : c'est à peu près la même chose, mais la liaison est détruite ou rendue inactive si la distance entre les particules devient trop grande ou si c'est la force produite par la liaison qui dépasse un certain seuil.
☞ permet de construire un modèle (très très élémentaire) de fracture.
 - à partir de là, on invente ce que l'on veut....

Quelques cas particuliers de liaisons

Avec le système précédent, il semble difficile d'intégrer quelque chose d'aussi simple qu'une force de gravité s'appliquant sur chaque particule : la force produite ne dépend pas de l'état de la particule, et surtout la liaison n'aurait qu'un seul point M de connexion.

Voici plusieurs options, de la moins bonne à la moins mauvaise.

- **la particule "à gravité intégrée"** :
cette méthode, très classique et très tentante (parce que très simple) consiste à ajouter cette force extérieure constante directement dans l'algorithme d'intégration (étape 1 - explicite) :
 $\vec{V}(t+1) = (\vec{V}(t) + \frac{h}{m}(\vec{F}(t) + \vec{G}))$ où $\vec{F}(t)$ est le "buffer" d'accumulation de force de la particule M, alimenté par les liaisons connectées à M)
Cette méthode **n'est pas conseillée** car elle rompt la "modularité" et la "physicalité" du modèle :
une particule seule et isolée ne doit subir aucune contrainte.

(3). cf. [wikipedia: loi de Hook](#) ou [wikipedia: déformations élastiques](#)

- le module <Link> "force externe" :

consiste à créer une liaison à peu près standard, dont le seul rôle est de distribuer une force constante (c'est le cas de la gravité) ou "pilotée" par une fonction externe (pour un modèle de vent, par exemple).

Le seul point de vigilance est que cette force n'a *a priori* qu'un seul point d'application dans le système. Il suffit alors de faire pointer l'autre vers une masse "poubelle" (NULL), ou vers un dispositif interactif (position/vitesse de la souris, joystick...).

C'est le modèle conseillé dans un premier temps pour les TP.

Principal défaut de cette méthode, pour un modèle de gravité ou de vent : sur un système à n particules mobiles, il faut créer n liaisons de ce type (une par particule).

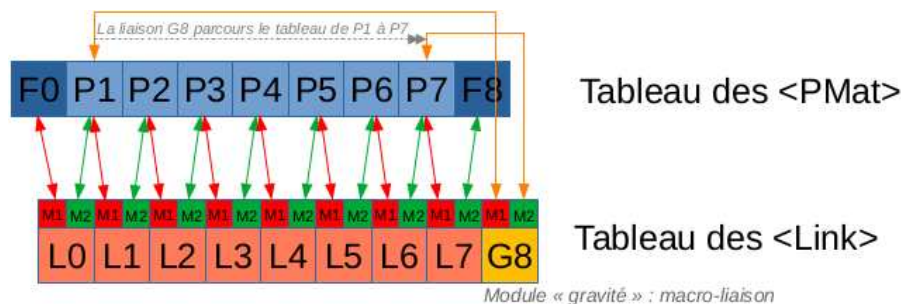
Ce type de module reste néanmoins **très** pratique pour interagir avec le système.

- le module <Link> "macro-liaison" :

c'est le plus pratique et le plus économique, même si il représente une entorse aux principes de base. Il consiste à créer une liaison connectée à un ensemble de particules (tout ou partie du tableau contenant tous les <PMat>). C'est donc la liaison elle-même qui "choisit" les particules sur lesquelles elle agit.

Concrètement ça ressemblerait à quelque-chose comme ça, pour une modèle de gravité simple :

- connexions : $M_1 = 1^o$ particule, $M_2 =$ dernière particule
- algorithme :
 - | $P = M_1$;
 - | tant que ($P < M_2$) faire :
 - | { $\vec{F}_P + = \vec{G}$; $P++$; }



structure algorithmique de "corde" avec macro-liaison "gravité"

- le module <Link> "spécial particules" :

La "combine" précédente (macro-liaison) est **très** pratique pour modéliser un système de particules (pas de topologie fixe)

Il suffit de définir **une seule liaison** de type *ressort-frein-conditionnel* (cf. plus loin) et de lui faire parcourir l'ensemble des paires de particules (complexité en n^2).

Si une structure accélératrice (pavage de l'espace, *quadtree/octree* ou autre) est présente, c'est cette liaison qui la gère $\Rightarrow$ transparent pour le simulateur.

Mise en oeuvre ()

Concrètement, un système masse-ressort s'écrit très simplement, via 2 modules : un pour les particules (PMat.[h|c]), l'autre pour les liaison (Link.[h|c]).

Voici un squelette, en C, de ces modules, à adapter (types Point, Vecteur, distances...) selon la dimension 1d, 2d ou 3d).

Cette version ne prend pas en compte les aspect graphiques, a priori totalement indépendants.

① les particules

```
<PMat.h>

typedef struct _ptm_
{
    float m;      //masse
    Point pos;    //position
    Vect vit;     //vitesse
    Vect frc;     //buffer d'accumulation des forces
    //mise à jour : intégrateur (h: pas de calcul)
    void (*update)(struct _ptm_ *this, float h);
} PMat;

// Constructeur
void M_builder(PMat *M, int type, float m, Point PO, Vect VO);

<PMat.c>

#include <PMat.h>
...
// Les "moteurs" : mise à jour de l'état
// intégrateur Leapfrog
static void update_leapfrog(PMat *M, float h)
{
    M->vit += h*M->frc/M->m; // intégration 1 : vitesse m.F(n) = (V(n+1)-V(n))/h -EXplicite
    M->pos += h*M->vit;       // intégration 2 : position V(n+1) = (X(n+1)-X(n))/h -IMplicite
    M->frc = 0.;              // on vide le buffer de force
}

// intégrateur Euler Explicite
// juste pour l'exemple : méthode très instable -> à éviter
// simple échange des 2 premières lignes par rapport à Leapfrog -> ça change tout
static void update_euler_exp(PMat *M, float h)
{
    M->pos += h*M->vit;       // intégration 1 : position V(n) = (X(n+1)-X(n-1))/h -EXplicite
    M->vit += h*M->frc/M->m; // intégration 2 : vitesse m.F(n) = (V(n+1)-V(n))/h -EXplicite
    M->frc = 0.;              // on vide le buffer de force
}

// mise à jour point fixe : ne fait rien
static void update_fixe(PMat *M, float h)
{
    // position et vitesse restent inchangées
    M->frc = 0.;              // on vide le buffer de force (par sécurité)
}
...
extern void M_builder(PMat *M, int type, float m, Point PO, Vect VO) // {
    M->m = m; // masse
    M->pos = PO; // position initiale
    M->vit = VO; // vitesse initiale
    M->frc = 0; // JAMAIS de force à la création
    switch (type) // choix de la fonction de mise à jour
    {
        case 0 : M->update = update_fixe; break; // Point Fixe
        case 1 : M->update = update_leapfrog; break; // Particule Leapfrog
        case 2 : M->update = update_euler_exp; break; // Particule Euler Exp.
    }
}
```

② les liaisons

```
<Link.h>

#include <PMat.h>
typedef struct _lnk_
{
    float k,l,z,s...; // paramètres divers
    PMat *M1,*M2;      // points M de connexion
    void (*update)(struct _lnk_ *this); //mise à jour : calcul/distrib des forces
} Link;
// Connecteur : branche L entre M1 et M2
void Connect(PMat *M1, Link *L, PMat *M2);
// Constructeurs
void Hook_Spring(Link *L, float k);
void Kinetic_Damper(Link *L, float z);
void Damped_Hook(Link *L, float k, float z);
void Cond_Damped_Hook(Link *L, float k, float z, float s);
...
// autres constructeurs.... autant qu'on veut
```

```
<Link.c>

#include <Link.h>
// Connecteur : branche L entre M1 et M2
extern void Connect(PMat *M1, Link *L, PMat *M2)
{
    L->M1 = M1;
    L->M2 = M2;
    // longueur "à vide" (ressort et assimilés) :
    L->l0 = distance(M1->pos,M2->pos);
}
// Les "moteurs" : calcul et distribution des forces
// Ressort linéaire
static void update_Hook(Link *L)
{
    float d = distance(L->M1->pos,L->M2->pos); // distance courante  $||\overrightarrow{M_1M_2}||$ 
    Vect u = Vecteur(L->M1->pos,L->M2->pos)/d; // direction  $M_1 \rightarrow M_2$ 
    Vect F = -L->k*(d-L->l0)*u; // force de rappel
    L->M1->frc += F; // distribution sur M1
    L->M2->frc -= F; // distribution sur M2
}
// Frein cinétique linéaire
static void update_Damper(Link *L)
{
    Vect F = -L->z*(L->M2->vit-L->M1->vit); // force de freinage
    L->M1->frc += F; // distribution sur M1
    L->M2->frc -= F; // distribution sur M2
}
// Combinaison des 2 (montage en parallèle)
static void update_Damped_Hook(Link *L)
{
    float d = distance(L->M1->pos,L->M2->pos); // distance courante  $||\overrightarrow{M_1M_2}||$ 
    Vect u = Vecteur(L->M1->pos,L->M2->pos)/d; // direction  $M_1 \rightarrow M_2$ 
    Vect F = -L->k*(d-L->l0)*u -L->z*(L->M2->vit-L->M1->vit); // force combinées
    L->M1->frc += F; // distribution sur M1
    L->M2->frc -= F; // distribution sur M2
}
// Liaison Ressort-Frein conditionnelle
// avec L->s=1. : simple "choc" visco-élastique
// avec L->s>1. : lien "inter-particule" - légère adhérence pour  $d \in [l_0, s * l_0]$ 
static void update_Cond_Damped_Hook(Link *L)
{
    float d = distance(L->M1->pos,L->M2->pos); // distance courante  $||\overrightarrow{M_1M_2}||$ 
    if (d>L->s*L->l0) return;
    Vect u = Vecteur(L->M1->pos,L->M2->pos)/d; // direction  $M_1 \rightarrow M_2$ 
    Vect F = -L->k*(d-L->l0)*u -L->z*(L->M2->vit-L->M1->vit); // force combinées
    L->M1->frc += F; // distribution sur M1
    L->M2->frc -= F; // distribution sur M2
}
...
```



```

<Link.c> (suite)
...
// Les constructeurs : choix de la nature de la liaison
// Ressort linéaire
extern void Hook_Spring(Link *L, float k)
{
    L->k = k;
    L->update = update_Hook;
}
// Frein cinétique linéaire
extern void Kinetic_Damper(Link *L, float z)
{
    L->z = z;
    L->update = update_Damper;
}
// Combinaison des 2 (montage en parallèle)
extern void Damped_Hook(Link *L, float k, float z)
{
    L->k = k;
    L->z = z;
    L->update = update_Damped_Hook;
}
// Liaison Ressort-Frein conditionnelle
extern void Cond_Damped_Hook(Link *L, float k, float z, float s)
{
    L->k = k;
    L->z = z;
    L->s = s;
    L->update = update_Cond_Damped_Hook;
}
...

```

③ Construction du modèle

C'est la fonction qui définit la nature et la position initiales des particules, la nature des liaisons et les connexions.

- fixe le pas d'échantillonnage h
- crée deux tableaux `PMat TabM[nbm]` ; pour les masses et `Link TabL[nbl]` ; pour les liens.
- initialise les masses (nature (fixe ou mobile), paramètre, positions...)
- initialise les liaisons (nature, paramètres)
- met en place la topologie éventuelle (connexions $M_a \leftrightarrow L \leftrightarrow M_b$) et initialise les longueurs à vide l_0 des liaisons.

👉 à ce stade, le système défini **doit** être à l'équilibre (aucune circulation de force).

👉 met en place les contraintes extérieures qui viendront perturber le système (mise hors d'équilibre).

④ Boucle de Simulation

③ moteur physique

Pour chaque pas de simulation (Fréquence $F_e = 1./h$) :

- ① Exécute en masse les méthodes `update` des liaisons indépendamment de leur nature :
`for (Link* L=TabL; L<TabL+nbl; L++) L->update(L);`
👉 après cette étape, plus aucune force ne doit circuler dans le système.
- ② Exécute en masse les méthodes `update` des particules indépendamment de leur nature :
`for (PMat* M=TabM; M<TabM+nbm; M++) M->update(M,h);`
👉 toutes les vitesses et positions ont été mises à jour, et les *buffer de forces* vidés.

⑥ moteur de rendu

Bien souvent on n'affiche pas tous les échantillons calculés par le moteur physique. Il est donc bon de disposer d'une fréquence d'affichage F_a indépendante.

Pour chaque pas de simulation (Fréquence $F_a \leq F_e$) :

Le moteur d'affichage s'appuie sur les états des particules (positions, vitesses) et éventuellement sur les données de connexion (liaisons) pour construire un modèle de rendu cohérent (mais largement indépendant) : facétisation, calcul de normales, surfaces implicite, *ray-tracer*....

Quelques "principes" à respecter

Paramétrage

On ne s'intéressera ici qu'à des modèles très simples (masses et liaisons toutes identiques, limitées dans un premier de temps à de simples "ressorts-amortis").

- Nombre de particules mobiles = Nombre de modes de déformation (cf. Cours Chap.4 - modèle modal)

Quelle que soit la topologie de connexion, un système à n masses libres se décompose en n modes de déformation (indépendants en 1D) : le *modèle modal*.

En 2D ou 3D, ces modes ne sont plus réellement indépendants (interdépendance des dimensions dues au calcul de distances) mais le principe général reste valable.

🔊 **Ce qu'il faut retenir** : augmenter le nombre de masses augmente le nombre de modes et donc le risque que les modes les plus élevés passent en zone de divergence (cf. Cours p.68).

- le paramètre de masse m n'est pas très utile en première approche

🔊 mettre toutes les masses à $m = 1$.

- les paramètres de raideur k et amortissement z des liaisons "ressort-amorti" sont liés au pas d'échantillonnage $h = 1./F_e$ (cf. Cours p.46 - paramètres réduits) : pour garder une "cohérence" physique à la simulation, il faut ajuster conjointement les 3 paramètres en gardant "constants" les paramètres réduits ($K_a = h^2 \frac{k}{m}$) et ($Z_a = h \frac{z}{m}$).

🔊 **Ce qu'il faut retenir** : il est pratique de paramétrer les modèles de la manière suivante :

- fixer h (ou F_e)
- fixer K_a, Z_a (et $M_a = m = 1$).
- en déduire les paramètres des modules à simuler : ($k = K_a * m * F_e^2$) et ($z = Z_a * m * F_e$)

attention : ces relations sont établies pour des modèles 1D linéaires. Elles peuvent s'appliquer également en 2D et 3D mais la non linéarité (dimensions "liées") de ces modèles peuvent introduire des écarts de comportement.

🔊 Un jeu de valeur fiable pour démarrer : $k \in [0.01, 0.1]$ et $z \in [0.0, 0.01]$ ($k/z \approx 10$) pour une structure de type "corde 1d" et $k \in [0.05, 0.5]$ et $z \in [0.0, 0.1]$ pour une structure de type "drapeau 3d" (puis ajustement avec F_e – et toujours $m = 1$).

- **divergence** : cela arrive très souvent !

Lorsqu'une simulation diverge, cela peut avoir plusieurs origines :

- tout explose dès le départ : cause la plus probable, le paramètre de viscosité est trop élevé.
Essayer d'abord avec $z = 0$., si ça ne diverge pas, augmenter et observer.

- ça diverge avec des oscillations : cause la plus probable, le paramètre de raideur est trop élevé ou la fréquence de calcul ($F_e = 1/h$) est trop faible.
- rien n'y fait, ça explose : un bug dans le code.... là, il n'y a plus qu'à le trouver et le corriger !

De manière générale :

- commencer par un modèle TRES simple (deux masses, un ressort), trouver une zone de paramètres à peu près stable, puis construire petit à petit en ajustant les paramètres.
- **autre point à retenir** : si le nombre de modes de déformation ne dépend que du nombre de particules, la distribution de ces modes dépend de la topologie de connexion.

Pour un même nombre de particules, plus la topologie est "dense" (beaucoup de liaisons), plus les modes seront rapprochés. Cela "abaisse" le mode le plus élevé qui est aussi le premier à provoquer la divergence (cf. Cours p.65-70).

👉 le bon paramétrage des modèles est, de loin, la partie la plus délicate dans l'usage des modèles physiques. C'est d'autant plus difficile que les meilleurs résultats sont obtenus en bordure des zones de stabilité (cf. Cours p.41).